

Nettoyer un fichier sale

le workflow pandas



Un seul fichier client, sale de bout en bout, nettoyé colonne par colonne : le code exact, le piège à chaque étape, le résultat attendu.

Tu viens de recevoir un export CRM B2B de 9 colonnes, et rien dedans n'est propre : doublons, casse incohérente, dates mélangées, montants au format français, libellés pays qui partent dans tous les sens. Ce guide reprend ce fichier fictif du début à la fin, dans l'ordre où un analyste le lirait le jour de la réception, pour que tu aies un réflexe transférable sur le prochain fichier sale que tu recevras vraiment.

1. Étape 0 : charger et ausculter le fichier avant d'y toucher

Cas : Le fichier `clients_export_juillet.csv` arrive tel quel dans ton dossier. Avant de corriger quoi que ce soit, tu dois savoir ce qui est cassé : combien de colonnes sont vraiment du texte, combien de valeurs manquent, combien de lignes sont dupliquées. C'est le diagnostic, pas encore le nettoyage.

```
import pandas as pd

df = pd.read_csv("clients_export_juillet.csv", dtype=str)

df.info()
df.isna().sum()
df.duplicated().sum()
```

Piège

Un junior lance `df.describe()` en premier : sur 9 colonnes toutes en object, ça ne renvoie presque rien d'utile.

Sortie

9 colonnes en object, des NaN cachés dans 5 d'entre elles, et 3 lignes dupliquées détectées avant tout nettoyage.

Identité du client : id, entreprise, email.

Ici, une seule saisie mal faite crée un faux doublon ou un faux « client introuvable ».

2. Colonne `id_client` : doublons exacts et doublons partiels

Cas : La société Rivoli Consulting a été saisie deux fois par deux commerciaux différents : une fois sous `id_client = CL-1042`, une fois sous `CL-1098`, avec le même `nom_entreprise` et le même `email`. Un simple `drop_duplicates()` ne verra jamais ce doublon, puisque la ligne n'est identique sur aucune colonne.

```
df.duplicated().sum()
df = df.drop_duplicates()

df.duplicated(subset=["nom_entreprise", "email"]).sum()
df = df.drop_duplicates(subset=["nom_entreprise", "email"], keep="first")
```

Piège

`drop_duplicates()` sans `subset` compare toutes les colonnes : deux lignes qui ne diffèrent que par `id_client` ne sont jamais fusionnées.

Sortie

Table dédoublée : le compteur de lignes passe de 214 à 211 après le `subset` sur nom et email, 3 doublons de saisie éliminés.

3. Colonne `nom_entreprise` : espaces et casse incohérente

Cas : Le même client apparaît écrit " Acme corp ", "ACME CORP" et "Acme Corp" selon la ligne. Un `groupby("nom_entreprise")` compte alors 3 clients au lieu d'1, et un total de chiffre d'affaires par client sort faux sans qu'aucune erreur ne s'affiche.

```
df["nom_entreprise"] = df["nom_entreprise"].str.strip().str.title()
```

Piège

`str.strip()` seul retire les espaces mais laisse "ACME CORP" et "Acme Corp" comme deux libellés différents.

Sortie

Un seul libellé "Acme Corp" pour les 3 lignes, un `groupby` qui compte enfin 1 client.

4. Colonne `email` : format invalide et casse PIÈGE CLASSIQUE

Cas : Le contact de Rivoli Consulting est saisi " Jean.Dupont@ACME.COM ". Trois semaines plus tard, tu fais un `merge` avec la table de facturation où le même email est écrit "jean.dupont@acme.com". Le `merge` ne trouve pas de correspondance : le client a l'air « introuvable », alors que c'est la même personne.

```
df["email"] = df["email"].str.strip().str.lower()

df["email_valide"] = df["email"].str.contains(
    r"^[^@\s]+@[^@\s]+\.[^@\s]+$", regex=True, na=False
)
```

Piège

Comparer deux emails sans `strip()` ni `lower()` donne un faux « non trouvé » dans un `merge`, jamais une erreur.

Sortie

Emails normalisés, plus une colonne `email_valide` qui flague les formats cassés via une regex.

À retenir : les 3 réflexes texte

Sur toute colonne texte d'un fichier sale, applique ce trio avant de croire qu'elle est propre.

1. Nettoyer

`str.strip()` pour les espaces parasites, jamais suffisant seul.

2. Harmoniser

`str.lower()` ou `str.title()` pour la casse, séparément.

3. Dédupliquer

`drop_duplicates(subset=[...])` sur les colonnes clés métier.

Dates et montants : le piège silencieux.

Ici, aucune erreur ne s'affiche : le script tourne, mais le chiffre est faux.

5. Colonne `date_signature` : formats mélangés ÉLIMINATOIRE

Cas : La ligne 12 porte "03/04/2024", la ligne 45 porte "2024-11-02" : deux formats dans la même colonne, un export CRM classique quand plusieurs commerciaux saisissent à la main. "03/04/2024" est ambigu : le 3 avril en France, le 4 mars pour un moteur anglo-saxon.

```
df["date_signature"] = pd.to_datetime(
    df["date_signature"], format="mixed",
    dayfirst=True, errors="coerce"
)

df["date_signature"].isna().sum()
```

Piège

Sans `dayfirst=True`, `format="mixed"` interprète "03/04/2024" comme le 4 mars, une inversion jour/-mois silencieuse, sans aucune erreur.

Sortie

Colonne en `datetime64[ns]`, et `NaT` pour les 2 dates réellement invalides, repérables par `isna()`.

6. Colonne `montant_contrat` : nombres au format français

Cas : Le montant est saisi "1 200,50 €" : espace insécable comme séparateur de milliers, virgule décimale, symbole euro collé. Un `pd.to_numeric` lancé directement sur cette chaîne casse tout de suite : pandas ne reconnaît ni l'espace, ni le symbole, ni la virgule comme séparateur décimal.

```
df["montant_contrat"] = (
    df["montant_contrat"]
    .str.replace(r"[\s\xa0]", "", regex=True)
    .str.replace("\u20ac", "", regex=False)
    .str.replace(",", ".", regex=False)
)
df["montant_contrat"] = pd.to_numeric(
    df["montant_contrat"], errors="coerce"
)
```

Piège

Appeler `pd.to_numeric` directement sur "1 200,50 €" lève une `ValueError` : impossible de convertir la chaîne en nombre.

Sortie

Colonne `float64` propre, somme vérifiable avec `df["montant_contrat"].sum()`.

Avant / après : 3 lignes extraites du fichier

Le nettoyage rend le montant lisible et calculable, sans changer la valeur réelle du contrat.

Brut (`montant_contrat`)

"1 200,50 €"
"980,00€"
"15000"

Nettoyé (`float64`)

1200.50
980.00
15000.00

Catégories et contacts : harmoniser sans perdre l'information.

Ici, le danger n'est plus l'erreur qui plante, c'est le filtre qui ment sans le dire.

7. Colonne `pays` : libellés incohérents

Cas : Le même pays apparaît sous 4 écritures : "France", "france", "FR", "Fr ance" (espace parasite de saisie). Un `groupby("pays")` compte alors 4 groupes au lieu d'1, et un rapport par pays sous-évalue chaque ligne.

```
mapping_pays = {"FR": "France", "FRANCE": "France"}

df["pays"] = (
    df["pays"].str.strip().str.upper()
    .str.replace(" ", "", regex=False)
)
df["pays"] = df["pays"].map(mapping_pays).fillna(df["pays"])
```

Piège

df["pays"].astype("category") seul ne fusionne rien : il crée 4 catégories distinctes à partir des 4 libellés bruts, doublons compris.

Sortie

Une seule valeur canonique "France", un groupby qui compte enfin 1 groupe.

8. Colonne statut_abonnement : casse variable et valeurs manquantes PIÈGE CLASSIQUE

Cas : Une quinzaine de lignes ont un statut_abonnement vide (NaN). Un collègue filtre avec df[df.statut_abonnement == "actif"] pour compter les clients actifs, persuadé d'avoir traité toute la table.

```
df["statut_abonnement"] = (
    df["statut_abonnement"].str.strip().str.lower()
)
df["statut_abonnement"] = df["statut_abonnement"].fillna(
    "non renseigné"
)

df["statut_abonnement"].value_counts()
```

Piège

NaN == "actif" renvoie False : le filtre exclut les lignes vides sans erreur ni avertissement, un faux négatif silencieux.

Sortie

Statuts nettoyés et un comptage explicite : actif 142, inactif 51, non renseigné 15.

9. Colonne telephone : formats mélangés

Cas : Le même numéro de Rivoli Consulting est écrit 3 façons : "01.42.55.67.89", "01 42 55 67 89", "0142556789". Le fichier source a été lu sans préciser le type, et un collègue a converti la colonne en entier pour « nettoyer », perdant le zéro initial.

```
df["telephone"] = (
    df["telephone"].astype(str)
    .str.replace(r"\D", "", regex=True)
)
```

Piège

Caster telephone en int fait perdre le zéro initial : "0142556789" devient 142556789, un numéro invalide.

Sortie

Colonne gardée en str, 10 chiffres homogènes pour les 3 écritures d'origine.

Pourquoi on revient toujours vérifier après un fillna ou un mapping

Un fillna ou un map qui tourne sans erreur ne veut pas dire qu'il a fait ce que tu croyais.

Sur ce fichier, un mapping_pays incomplet aurait laissé passer un libellé oublié sans le signaler : .map() renvoie NaN pour tout ce qui n'est pas dans le dictionnaire. Le réflexe qu'un junior saute : relancer value_counts() juste après chaque nettoyage catégoriel, pour vérifier que le nombre de valeurs distinctes correspond à ce que tu attendais.

10. Colonne `notes_commercial` : texte libre très incomplet

Cas : 70% des lignes n'ont pas de note commerciale (NaN). La colonne sert plus tard à un rapport de complétude par commercial : « combien de clients ont une note de suivi? »

```
df["notes_manquantes"] = df["notes_commercial"].isna()
df["notes_commercial"] = df["notes_commercial"].fillna("")
```

Piège

Faire le `fillna("")` avant de flaguer la complétude casse l'analyse : une chaîne vide et un NaN ne se comptent plus pareil ensuite.

Sortie

Colonne conservée, plus un flag `notes_manquantes` réutilisable pour tout rapport de qualité de saisie.

11. Étape finale : sauvegarder le fichier propre

Cas : Le fichier est nettoyé colonne par colonne. Il reste une dernière décision, la plus oubliée par un junior pressé : comment l'écrire pour que la prochaine relecture ne réintroduise pas de saleté.

```
df.to_csv("clients_export_juillet_propre.csv", index=False)
```

Piège

Oublier `index=False` ajoute une colonne parasite `Unnamed: 0` à la relecture du CSV : elle pollue toute analyse suivante.

Sortie

Un fichier `clients_export_juillet_propre.csv` prêt à être rechargé sans colonne fantôme.

Fichier propre : le workflow qui se répète

Ce fichier CRM était fictif; le désordre qu'il contenait, non. C'est celui de n'importe quel export reçu un lundi matin.

À retenir : le workflow colonne par colonne

Applique cet ordre à tout fichier client sale que tu recevras, pas seulement à celui de ce guide.

1. Diagnostiquer

info, isna, duplicated avant tout.

2. Texte

strip puis lower/title, dédupliquer.

3. Dates / nombres

`to_datetime(dayfirst=True)`, retirer symboles.

4. Catégories

Mapping explicite, jamais `astype("category")` seul.

5. Sauvegarder

`to_csv(index=False)`, sinon colonne fantôme.

Réflexe

`value_counts()` après `fillna` ou mapping.

Fait main, en solo.

Si ce workflow t'a fait gagner du temps, partage-le à un futur collègue qui va recevoir son premier fichier sale. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

